

Estrutura de Dados II

Algoritmos de Ordenação:

- CountSort (Contagem)
- MergeSort
- QuickSort

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello world!" << endl;
    return 0;
}
```

Algoritmos de Ordenação

Alguns Algoritmos de Ordenação:

- Bubblesort
- Ordenação por Inserção
- Ordenação por Seleção
- **Ordenação por Contagem**
- **MergeSort**
- **QuickSort**

ORDENAÇÃO POR CONTAGEM



Algoritmo de Ordenação por Contagem

Esse algoritmo pode ser o mais rápido em vários casos, pois ele ordena os dados em **tempo linear**.

Essa ordenação pressupõe que os dados serão sempre uma entrada de **1 a n**, para qualquer inteiro **n**, ou seja, se terão 6 dígitos, a entrada precisa ser qualquer combinação possível de **6 números** que variam entre si de um a 6.

Algoritmo de Ordenação por Contagem

A base matemática do algoritmo está em determinar para cada elemento x de entrada o número de elementos menores que x , e depois inserir diretamente x na posição de saída na estrutura.

O índice indica o elemento e o valor da posição o numero de vezes que ele aparece.

Algoritmo de Ordenação por Contagem

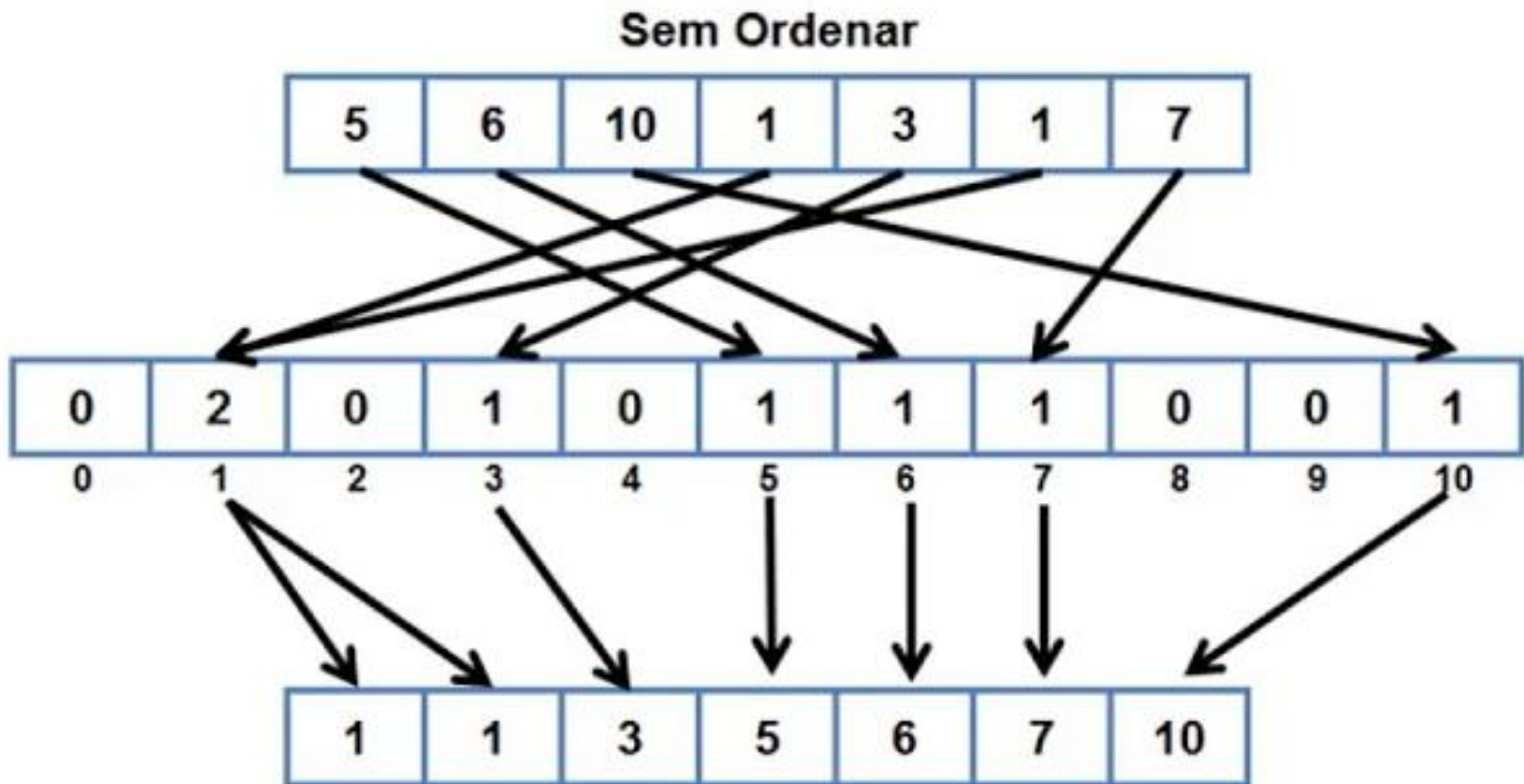
Exemplo: *O índice indica o elemento e o valor da posição o numero de vezes que ele aparece.*

Antes de ordenar: 8 1 2 5 1 4 2



Após ordenação 1 1 2 2 4 5 8

Ordenação por Contagem



Algoritmo de Ordenação por Contagem

```
#include <stdio.h>
#include <stdlib.h>
```

```
main ()
{
```

```
//Inicialização do Vetor
int v[7]={5,6,10,1,3,1,7};
int n=7;
int i, j = 0, aux;
```

```
//Algoritmo de Ordenação
counting_sort(v,n);
```

```
//Laço de impressão do Vetor
for (int q=0; q<n; q++)
{
    printf("%d ",v[q]);
}
}
```

```
void counting_sort(int *array, int size)
{
    int i, min, max;
    min = max = array[0];

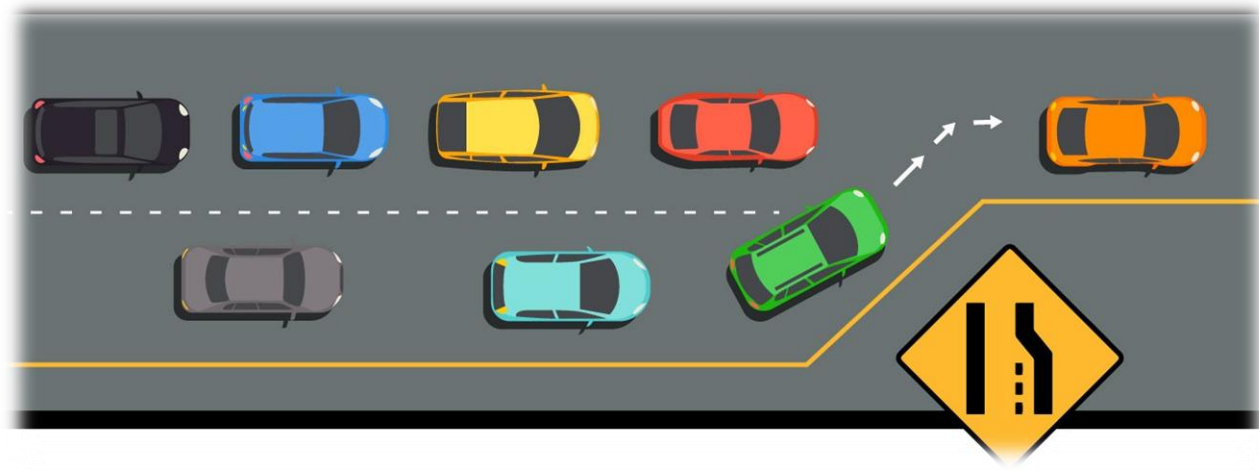
    //Identifica o Maior Elemento
    for(i = 1; i < size; i++)
    {
        if (array[i] < min)
            min = array[i];
        else if (array[i] > max)
            max = array[i];
    }
    int range = max - min + 1;
    int *count = (int *) malloc(range * sizeof(int));

    //Marca Todas as posições com Zero
    for(i = 0; i < range; i++)
        count[i] = 0;

    //Marca as posições ocupadas
    for(i = 0; i < size; i++)
        count[array[i] - min]++;

    int j, indice = 0;
    //Array recebe as posicoes ocupadas
    for(i = min; i <= max; i++)
        for(j = 0; j < count[ i - min ]; j++)
            array[indice++] = i;
    free(count);
}
```

ORDENAÇÃO MERGESORT



Algoritmo de ordenação Mergesort

Esse é um algoritmo que segue uma **técnica de dividir**, ordenar e juntar (**dividir para conquistar**).

A eficiência é a mesma para melhor, pior e caso médio. Independentemente de como os dados do *array* estão organizados a ordenação será eficaz. ($O(n \log_2 n)$)

O conceito para que ele funcione é o seguinte:

- Ordenar uma estrutura significa ordenar várias subestruturas internas já ordenadas.
- Caso essas estruturas não estejam ordenadas, basta ordená-las pelo mesmo método (ordenar suas subestruturas internas... até o infinito).
- Para ordenar subestruturas, basta dividir a estrutura atual e fazer uma chamada recursiva (**recomeçar com um problema mais simples**).

Algoritmo de ordenação Mergesort

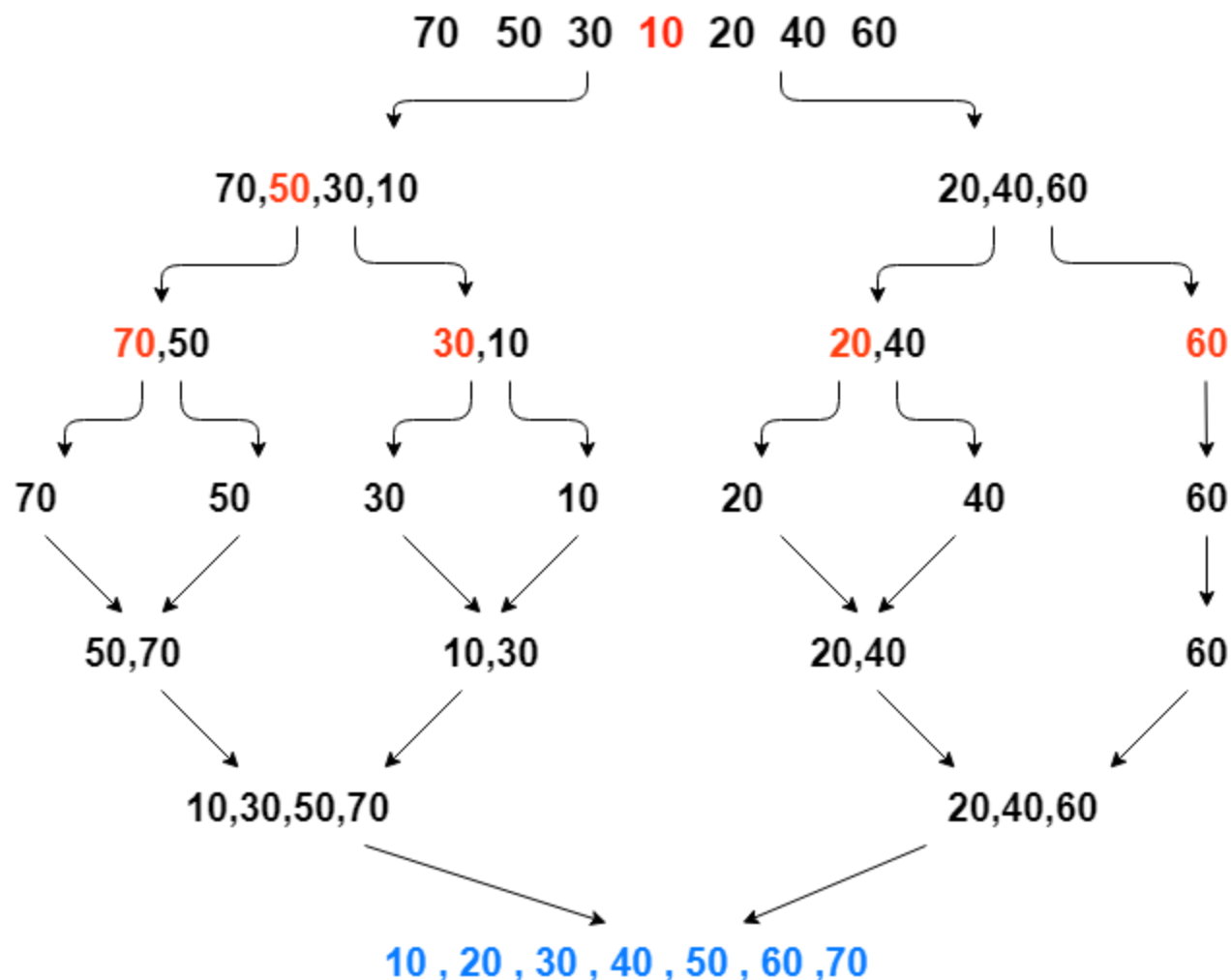
Os três passos do algoritmos que se aplicam ao ***mergesort*** são:

- **Dividir:** Calcula o ponto médio do sub-arranjo.
- **Conquistar:** Recursivamente resolve dois subproblemas, cada um de tamanho $n/2$, o que contribui com para o tempo de execução.
- **Combinar:** Unir os sub-arranjos em um único conjunto ordenado.

Exemplo:

- Ordenar 70, 50, 30, 10, 20, 40, 60... o algoritmo irá dividir em pares: {70,50}, {30,10}, {20,40}, {60}...
- Ordenados {50,70}, {10,30}, {20,40}, {60}.
- Depois ir fazendo o merge desses dados, ou seja, juntando-os em dois pares ordenados: {10,30,50,70}, {20,40,60},
- Depois juntar novamente: 10, 20, 30, 40,50, 60, 70.
- **No final do algoritmo a seqüência inicial está ordenada a partir de divisões.**

Ordenação Mergesort



Mergesort

```
void mergeSort(int vec[], int vecSize)
{
    int mid;
    if (vecSize > 1)
    {
        mid = vecSize / 2;
        mergeSort(vec, mid);
        mergeSort(vec + mid, vecSize - mid);
        merge(vec, vecSize);
    }
}
```

```
main ()
{
    //Inicialização do Vetor
    int v[7]= {70,50,30,10,20,40,60}, n=7;
    int i, j = 0, aux;

    //Algoritmo de Ordenação
    mergeSort(v,n);

    //Laço de impressão do Vetor
    for (int q=0; q<n; q++)
    {
        printf("%d ",v[q]);
    }
}
```

```
//Kernell do Algoritmo
void merge(int vec[], int vecSize) {
    int mid;
    int i, j, k;
    int* tmp;
    tmp = (int*) malloc(vecSize * sizeof(int));
    if (tmp == NULL) {
        exit(1); }
    mid = vecSize / 2;
    i = 0;
    j = mid;
    k = 0;
    while (i < mid && j < vecSize) {
        if (vec[i] < vec[j]) {
            tmp[k] = vec[i];
            ++i; }
        else {
            tmp[k] = vec[j];
            ++j; }
        ++k; }
    if (i == mid) {
        while (j < vecSize) {
            tmp[k] = vec[j];
            ++j;
            ++k; } }
    else {
        while (i < mid) {
            tmp[k] = vec[i];
            ++i;
            ++k; } }
    for (i = 0; i < vecSize; ++i) {
        vec[i] = tmp[i]; }
    free(tmp); }
```

ORDENAÇÃO QUICKSORT

$$a < \textcircled{b} < c$$

Algoritmo de Ordenação Quicksort

O Algoritmo Quicksort foi criado em 1960. É considerado um dos métodos de ordenação mais rápido para uma ampla variedade de situações, devido a complexidade $O(n^2)$ no pior caso e $O(n \log n)$ no melhor e médio caso.

O Algoritmo é implementado em várias linguagens, largamente utilizado/customizado em projetos de linguagens de programação o que o torna um dos mais utilizados.

A lógica do funcionamento do algoritmo baseia-se em uma rotina fundamental cujo nome é ***particionamento***.

Particionar significa escolher um número qualquer presente no conjunto de dados, chamado de ***pivot***, e colocá-lo em uma posição tal que todos os elementos à esquerda são menores ou iguais e todos os elementos à direita são maiores, **repetindo sucessivamente**.

Algoritmo de Ordenação Quicksort

O Quicksort também adota a estratégia de divisão e conquista.

Os passos são:

- a) Escolha um elemento da lista, denominado ***pivot (pivô)***;
- b) Rearranje a lista de forma que todos os elementos anteriores ao pivô sejam menores que ele, e todos os elementos posteriores ao pivô sejam maiores que ele. Ao fim do processo o pivô estará em sua posição final e haverá duas sublistas não ordenadas. **Essa operação é denominada *partição***;
- c) **Recursivamente** ordene a sublista dos elementos menores e a sublista dos elementos maiores.

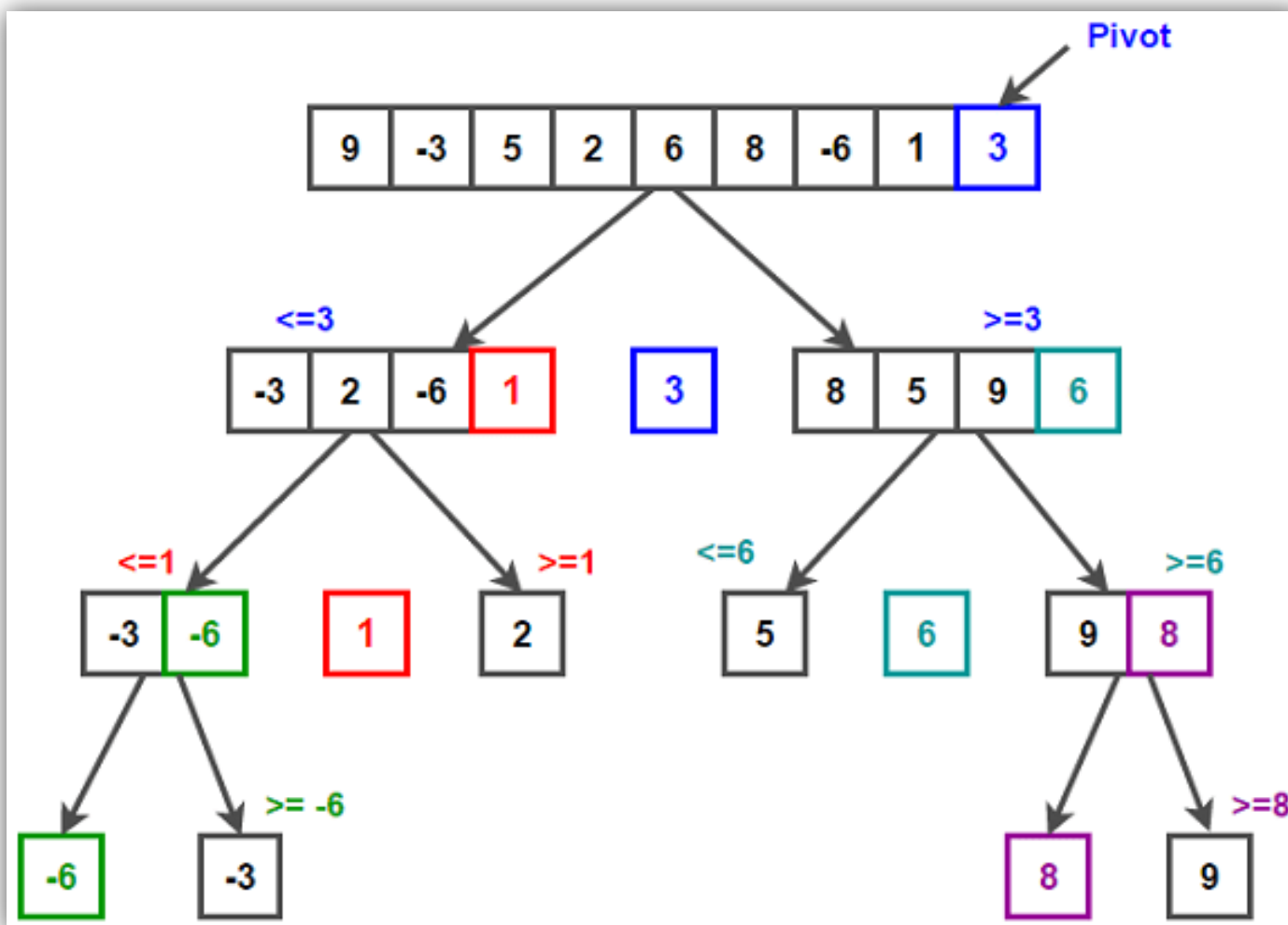
Algoritmo de Ordenação Quicksort

A base da recursão (**ponto de parada**) são as listas de tamanho zero ou um, que estão sempre ordenadas.

O processo é finito, pois a cada iteração pelo menos um elemento é posto em sua posição final e não será mais manipulado na iteração seguinte.

Apesar de estar na mesma classe de complexidade do **Merge Sort**, há experimentos que demonstram que **o Quick Sort em seu melhor caso e caso médio é por volta de 3x mais eficiente que o MergeSort**, porque ele contém constantes menores.

Ordenação Quicksort



Algoritmo de Ordenação Quicksort

```
main ()
{

    //Inicialização do Vetor
    int v[9]= {9,-3, 5, 2, 6, 8,
               -6, 1, 3}, n=9;
    int i, j = 0, aux;

    //Algoritmo de Ordenação
    quickSort(v,0,n);

    //Laço de impressão do Vetor
    for (int q=0; q<n; q++)
    {
        printf("%d ",v[q]);
    }
}
```

```
void invertre(int* a, int* b) {
    int tmp;
    tmp = *a;
    *a = *b;
    *b = tmp;
}

int partition(int vec[], int esq, int dir) {
    int i, j;
    i = esq;
    for (j = esq + 1; j <= dir; ++j) {
        if (vec[j] < vec[esq]) {
            ++i;
            invertre(&vec[i], &vec[j]);
        }
    }
    invertre(&vec[esq], &vec[i]);
    return i;
}

void quickSort(int vec[], int esq, int dir) {
    int r;
    if (dir > esq) {
        r = partition(vec, esq, dir);
        quickSort(vec, esq, r - 1);
        quickSort(vec, r + 1, dir);
    }
}
```